

SAHIL SHEORAN

Backend Developer | Python | Django & DRF | REST APIs | PostgreSQL | Celery | Redis | Docker

New Delhi, India • +91-8178328975 • sahilsheoran24@gmail.com

Portfolio: Sahil-coder50.github.io • GitHub: github.com/Sahil-coder50 • LinkedIn: linkedin.com/in/sahil-sheoran-py

PROFESSIONAL SUMMARY

Backend Developer with 10 months of production experience designing and shipping REST APIs in Python, Django, and Django REST Framework. Built and owned backend systems end-to-end — from database schema and RBAC-based auth to async pipelines with Celery and Redis, real-time notifications via WebSockets, and event streaming with Kafka. Maintains published open-source packages and project templates used to standardize authentication and file handling across projects. Comfortable making architecture decisions, optimizing PostgreSQL queries, and deploying containerized services with Docker across staging and production.

TECHNICAL SKILLS

Languages: Python

Backend Frameworks: Django, Django REST Framework (DRF), FastAPI

API Design & Auth: REST APIs, JWT, OAuth 2.0 (Google OAuth), Token-based Authentication, Role-Based Access Control (RBAC), Serializer Validation

Databases: PostgreSQL, SQL — schema design, query optimization, Django ORM

Caching & Session Store: Redis — caching, Celery task brokering

Async & Task Queues: Celery — background jobs, scheduled tasks, async email/notification pipelines

Event Streaming: Apache Kafka — event-driven architecture

Real-Time Communication: WebSockets via Django Channels — real-time push notifications, live status updates

Containerization: Docker, Docker Compose — local dev, staging, and production environment standardization

Engineering Practices: Git, GitHub, Audit Logging, N+1 Query Optimization (select_related / prefetch_related), Pagination, Agile, Package Publishing (PyPI), End-to-End Backend Deployment

PROFESSIONAL EXPERIENCE

Software Developer | Weboapp Discovery Private Limited

Noida & Panchkula, India | September 2025 – Present

Project Management System | September 2025 – November 2025

- Took ownership of a broken RBAC system and rebuilt it from static to dynamic permission handling — giving the team fine-grained, role-specific access control that could evolve without code changes.
- Diagnosed root causes of recurring API crashes and data inconsistencies, then fixed them systematically rather than patching symptoms.
- Designed and scheduled Celery background jobs with Redis to automate recurring tasks that had previously required manual intervention from the team.
- Plugged Apache Kafka into the project activity layer so cross-service events were streamed reliably, removing tight coupling between backend modules.
- Brought WebSockets (Django Channels) into the notification layer so project members were alerted to task updates in real time, without waiting for the next API poll.
- Containerized the backend with Docker and Docker Compose, making deployment to staging and production a repeatable, one-command process.
- Hardened the API layer by adding serializer-level validations that caught bad data at the boundary before it could corrupt the database.
- Partnered with frontend and QA teams during deployment cycles, treating API stability as a shared responsibility rather than a handoff.

CRM Backend System | November 2025 – Present

- Designed the entire PostgreSQL database schema from scratch for leads, contacts, accounts, and deals — making deliberate modeling decisions that kept relational queries clean and scalable as data grew.
- Owned the reporting and analytics API layer end-to-end, solving a hard frontend problem of dynamic grouping and filtering by building a flexible query engine that drove chart generation without hardcoding logic.

- Took initiative to build a field-level audit logging system on top of RBAC and token-based auth — going beyond the requirement to ensure every data change was traceable by role and user.
- Identified N+1 query bottlenecks across 15+ APIs and resolved them using `select_related` and `prefetch_related`, cutting unnecessary database hits for relational data fetches.
- Eliminated API blocking on email and notification delivery by designing a Celery-Redis async pipeline that offloaded all such workflows to background workers.
- Introduced Apache Kafka as an event streaming layer to decouple service communication — a decision that made the CRM pipeline resilient to spikes and easier to extend.
- Replaced polling-based updates with a WebSocket layer (Django Channels) so frontend clients received real-time notifications instantly, removing unnecessary API load.
- Drove the adoption of Docker and Docker Compose across the team, standardizing environments and eliminating the 'works on my machine' problem during staging and production deployments.
- Enforced dataset pagination across all reporting endpoints to prevent memory and timeout issues when querying large CRM record sets.

Ticket Management System | *March 2026 – Present*

- Identified limitations in Django's default admin history logging and rebuilt the audit system from scratch, giving the team flexible, application-specific change tracking they could actually rely on.
- Offloaded notification dispatch from synchronous ticket APIs by wiring Celery and Redis into the workflow, freeing up API threads and improving responsiveness under load.
- Removed the need for frontend polling by introducing WebSockets (Django Channels), so users saw ticket status changes the moment they happened — without a page refresh.
- Standardized the local and QA setup using Docker, cutting onboarding friction and ensuring the team always tested against an identical environment.
- Hunted down and fixed backend bugs across ticket workflows that were causing silent failures, bringing stability to a codebase the team had stopped trusting.

OPEN SOURCE CONTRIBUTIONS

- **auth-drf** (PyPI, self-published) — a Django REST Framework package providing JWT authentication, Role-Based Access Control, Google OAuth, and token refresh, removing the need to rewrite auth boilerplate on every project.
- **image-upload** (PyPI, self-published) — a lightweight FastAPI package for async file upload handling, validation, and absolute URL generation using Pydantic.
- **drf-project-template** & **fastapi-project-template** (GitHub) — reusable project scaffolds with pre-configured Celery, Redis, Docker, Alembic migrations, and structured exception handling, used to bootstrap new backend services without starting from a blank folder.

EDUCATION

Master of Engineering (M.Tech), Computer Science

SGT University | CGPA: 9.01 / 10

Bachelor of Technology (B.Tech), Computer Engineering

SGT University | CGPA: 8.00 / 10